

Module III — Development Environments & Technical Efficiency

How Modern Programmers Think

Cloud Contraptions LLC - www.cloudcontraptions.com

Overview

What This Module Covers

- The "how" of programming, not just the "why"
- Operating systems beneath your code editor
- Command-line tools and efficient workflows
- Professional code editors and keyboard shortcuts
- Research skills for independent problem-solving
- Builds the CivicTrack developer workspace

Learning Objectives

- Understand Windows, macOS, and Linux for development
- Work effectively at the command line
- Configure a professional development environment
- Use VS Code at an advanced level
- Research and learn independently and wisely
- Bridge existing professional skills to programming

Why It Matters & Key Principles

- Your environment is like a restaurant kitchen
- Efficiency compounds: seconds saved become hours
- Mastery takes practice until muscle memory forms
- "Learning to learn" is the core meta-skill
- Normalize looking things up constantly

Operating Systems Overview

What an Operating System Does

- Conducts the computer's hardware "orchestra"
- Process management: schedules CPU time
- Memory management: allocates and protects RAM
- File system: organizes stored files
- Handles devices, security, and permissions
- Your code always runs on an OS

Windows, macOS, and Linux at a Glance

Aspect	Windows	macOS	Linux
Desktop share	~71%	~20%	~8%
Server share	minority	negligible	most
Focus	Enterprise, .NET	Design, iOS	Servers, DevOps
Command line	PowerShell	Unix Terminal	Native shell

- macOS and Linux share Unix roots
- Unix skills transfer to production servers

Why Most Servers Run Linux

- Free: no per-server licensing costs
- Reliability: famous for years of uptime
- Open source: inspectable and fixable
- Historical: the web stack grew on Linux
- Unix philosophy: "do one thing well"
- Huge DevOps and sysadmin community

File Systems and Path Conventions

```
Windows: C:\Users\Sarah\Documents\my-app  
Unix:    /Users/sarah/Documents/my-app  
Home:    ~/Documents/my-app
```

- NTFS is case-insensitive; ext4 is case-sensitive
- Case mismatches cause "works on my machine" bugs
- Windows: backslashes and drive letters
- Unix: forward slashes from root `/`; `~` = home

Virtual Machines vs. Containers

- VMs simulate a whole computer; full isolation
- VMs are heavy: more resources, slower, large
- Containers share the host kernel; lightweight
- Docker bundles an app with its dependencies
- Containers start fast; run consistently anywhere

Windows Essentials

Why the Command Line Matters

- Faster than the GUI for many tasks
- Automate repetitive work with scripts
- Some operations exist only on the command line
- Consistent across machines and remote servers
- Essential to programmers, like spreadsheets to accountants

CMD, PowerShell, and Windows Terminal

- CMD: legacy DOS-era shell, rarely used today
- PowerShell: modern, object-oriented, recommended shell
- PowerShell Core runs on Windows, macOS, Linux
- Windows Terminal hosts multiple shells in tabs
- Focus your learning on PowerShell

Essential PowerShell Commands

```
Get-Location           # print current directory
Set-Location C:\projects # change directory
Get-ChildItem          # list files and folders
Get-ChildItem -Recurse -Filter *.js # find files
Select-String "TODO" app.js # search inside a file
Copy-Item a.txt b.txt   # copy a file
Remove-Item old.txt     # delete (no undo!)
```

- Pipe output between commands with `|`

Environment Variables and PATH

```
$env:PATH                # view the PATH variable
Get-ChildItem env:        # list all variables
$env:MY_VAR = "hello"    # set for this session
```

- PATH lists folders searched for commands
- Store secrets like API keys in variables, not code
- Restart the terminal after permanent changes

WSL: Linux on Windows

```
wsl --install # installs WSL and Ubuntu
```

- Runs a real Linux kernel on Windows
- Best of both: Linux tools plus Windows apps
- Windows files appear under `/mnt/c/` in WSL
- Ideal for web, DevOps, and container work

Installing Software with winget

```
winget install Python.Python.3.12  
winget install OpenJS.NodeJS  
winget install Git.Git  
winget install Microsoft.VisualStudioCode
```

- Package managers automate downloads and installs
- One command per package; no wizards
- Use full IDs to avoid ambiguity
- Chocolatey is a community alternative

Text Editors and IDEs

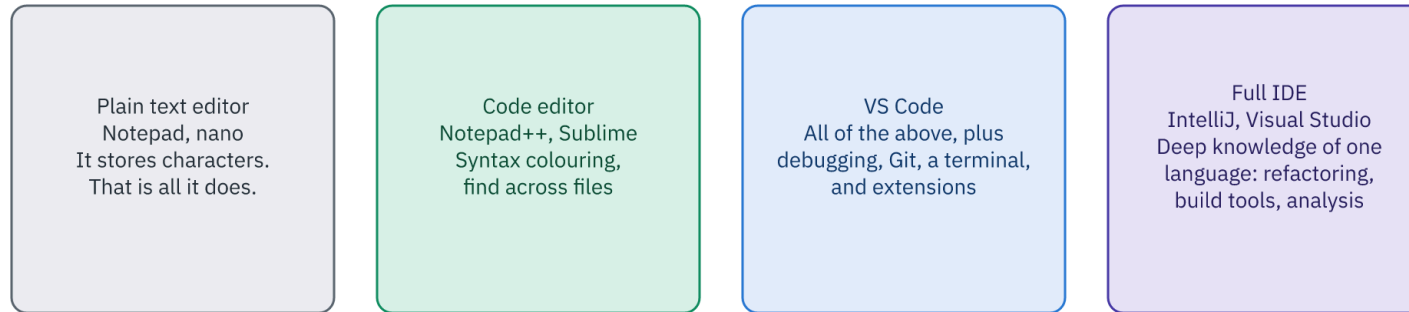
Editors vs. IDEs: The Spectrum

- Text editor: plain text only, like Notepad
- Code editor: highlighting and completion, like VS Code
- IDE: debugging, testing, project tools built in
- Word cannot handle code files properly
- Beginners: start with a code editor

The Spectrum, Drawn

From Text Editor to IDE

Not better and worse — a trade between how light the tool is and how much it does for you.



lighter, opens instantly, knows nothing about your code <<----->> heavier, slower to start, understands your code

This course uses VS Code because it sits in the middle: light enough to open for one file, capable enough to debug a real program — and it is free on every operating system.

You will meet all four in a career. Knowing which one a job needs is part of the skill.

VS Code: The Industry Standard

- Free, open source, and cross-platform
- Lightweight: starts in seconds
- Built-in Git integration and terminal
- Thousands of extensions add functionality
- IntelliSense gives smart auto-completion
- Massive community and documentation

Essential VS Code Features

- Syntax highlighting plus IntelliSense auto-completion
- Extensions: Prettier, GitLens, Python, ESLint
- AI assistants suggest code; verify before trusting
- Command Palette (Ctrl+Shift+P) runs any command
- Integrated terminal keeps code and shell together
- Workspace settings shareable via version control

Productive Keyboard Shortcuts

Ctrl+P	Open file by name
Ctrl+Shift+P	Command Palette
Ctrl+/ Ctrl+D	Comment / uncomment line Select next matching word
Alt+Up / Alt+Down	Move line up or down
Ctrl+F / Ctrl+H	Find / find and replace

- Small time savings compound across a career
- Multi-cursor edits many places at once

Smart Research Practices

Programming Is 50% Searching

- Experienced developers look things up daily
- Technology is too vast to memorize fully
- Looking up beats misremembering; it is faster
- Average programmer searches 10-20 times daily
- Searching effectively is a professional skill

Effective Google Search Operators

```
site:python.org list comprehension    # official docs
"exact error message"                 # match a phrase
filetype:pdf                           # only PDFs
-exclude                               # drop a keyword
```

- Paste exact error messages first
- Then official docs, then tutorials
- Fall back to the general concept last

Evaluating Sources You Find

- Official docs: most accurate and current
- Stack Overflow: check votes, date, and comments
- The accepted answer is not always best
- Blogs and tutorials: good for the "why"
- Video: shows workflow but slow to search
- Always check how recent the source is

Asking Good Questions & Using AI Wisely

- State the real problem, not a confusing goal
- Include a minimal reproducible example
- Paste the full error; say what you tried
- Search before asking; respect people's time
- Treat AI as a sometimes-wrong colleague
- Verify AI output against docs, then run it

Leveraging Online Coding Resources

The Learning Resources Ecosystem

- Official docs: most authoritative source
- Interactive platforms: hands-on practice
- Tutorials and blogs: explanation-focused
- Communities and open source: peer learning
- No single category is best; mix them

Official Docs & Interactive Platforms

Resource	Best For
Official docs	Accurate technical reference
freeCodeCamp	Free beginner web courses
Codecademy	Structured interactive lessons
LeetCode / HackerRank	Algorithm practice
Exercism	Human feedback on your code

- Practice on platforms, then build real projects

Communities and Open Source

- Stack Overflow: searchable Q&A archive
- Reddit r/learnprogramming: beginner-friendly discussion
- Discord servers: real-time community help
- Read open source to learn real structure
- Start with "good first issue" contributions
- Read the README, then follow the code

Building Your Learning Curriculum

- Define a specific goal, not "learn programming"
- Phase 1: fundamentals via one interactive course
- Phase 2: tutorials and small projects
- Phase 3: build a project you care about
- 80% of learning comes from building
- Commit to one resource before switching

Wrap-Up

Key Takeaways

- The OS shapes how and where code runs
- Linux dominates servers; Unix literacy is essential
- The command line brings speed and automation
- Master VS Code; shortcuts compound over time
- Looking things up is normal, professional work
- Verify every source, including AI, against docs

Discussion Questions

- Why does Linux dominate servers but not desktops?
- How does WSL change Windows as a dev platform?
- When would you choose containers over virtual machines?
- What is your strategy for searching a cryptic error?
- How do you judge whether a tutorial is trustworthy?